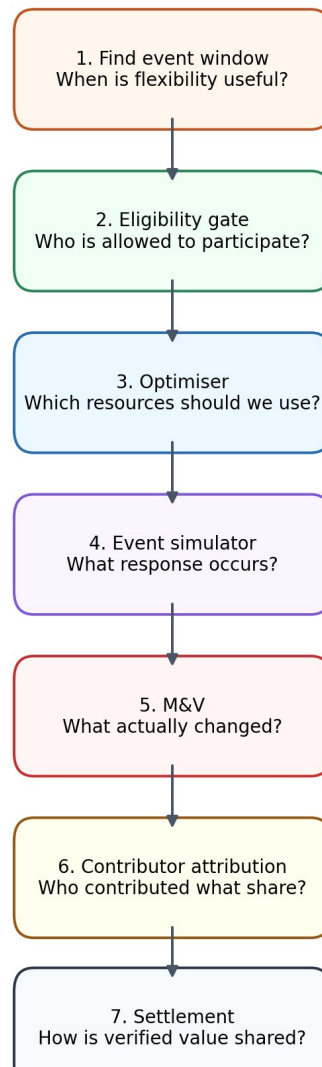


SUNSHINE WALLET

Complete Engine Flow

Detailed logic, calculations, data contracts, simulation rules, M&V, contributor attribution and settlement

Sunshine Wallet Engine - End-to-End Logic



Core idea

The engine decides when an event is useful, which resources may help, what response actually occurred, how much of that response is defensible, and how the resulting value is shared. It does not trace individual electrons from one household to another.

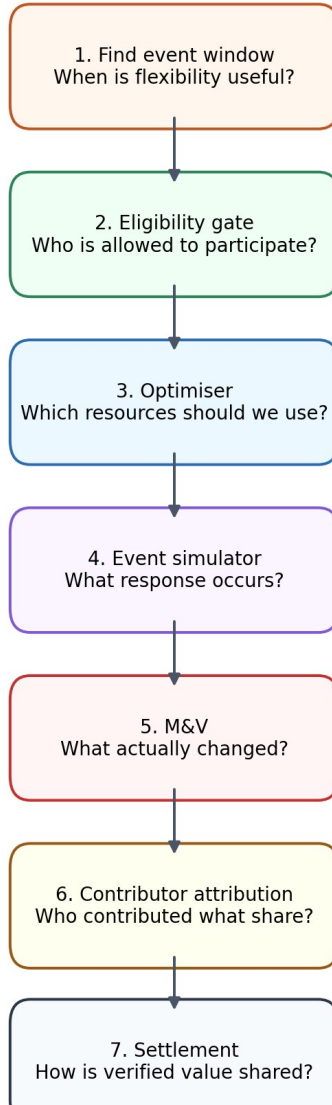
Contents

1. Engine in one page
2. What the engine is and is not
3. Inputs and data honesty
4. Algorithm 1 - Event-window selection
5. How the target flexibility is calculated
6. Algorithm 2 - Resource eligibility gate
7. Algorithm 3 - Resource scoring and optimisation
8. Algorithm 4 - Deterministic event simulation
9. Algorithm 5 - Measurement & Verification (M&V)
10. Baseline design in detail
11. M&V confidence and settlement gate
12. Algorithm 6 - Contributor attribution
13. Algorithm 7 - Settlement and Equity Floor
14. Resident, contributor and Council flows
15. Recommended MVP data model
16. JSON/API contract examples
17. Coordinator function and pseudocode
18. Test scenarios
19. Known limitations and defensible judge language
20. Implementation priorities and cheat sheet

1. Engine in one page

The Sunshine Wallet engine can be remembered as seven steps. The order matters because settlement should only happen after the event has been verified.

Sunshine Wallet Engine - End-to-End Logic



Step	Question answered	MVP output
1. Window selection	When should a Sunshine Event run?	Chosen time window or NO EVENT
2. Eligibility	Which resources are technically and ethically allowed?	Eligible + rejected resources with reasons
3. Optimiser	Which eligible resources should we use?	Dispatch plan that meets the target
4. Simulation	What response occurs after dispatch?	Deterministic simulated meter/device response
5. M&V	What changed versus no	Verified flexibility + confidence

Step	Question answered	MVP output
	intervention?	
6. Contributor attribution	What share of qualifying contribution belongs to each contributor?	Contribution shares / attributed kWh
7. Settlement	How is verified value shared?	Equity credits, contributor rewards, reserve

Important ordering correction

The product story may be described as Find - Catch - Share - Prove, but the software should verify before final settlement: Find -> Catch/Optimise -> Run -> Prove/Verify -> Share/Settle.

2. What the engine is - and is not

2.1 What it does

- Uses forecast/network inputs to find a useful local flexibility window.
- Checks device consent, availability, compatibility, location and comfort constraints.
- Ranks eligible flexible resources and selects enough energy to meet a target.
- Simulates the physical response for the hackathon using transparent deterministic factors.
- Estimates a baseline and calculates how much event-period demand changed.
- Attributes contributor shares using metered/accounting quantities rather than electron tracing.
- Converts verified value into bill/program credits and contributor rewards under an Equity Floor.

2.2 What it does not claim

- It does not route Maria's exact electrons directly to Dinh.
- It does not assume every kWh exported at midday was "rescued" by Sunshine Wallet.
- It does not directly control real smart meters, batteries or hot-water circuits in the hackathon.
- It does not claim the prototype settlement rate is a real tariff or market price.
- It does not assume a suburb boundary is the same as the relevant electrical network zone.

3. Inputs and data honesty

For the 25-hour hackathon, most grid and device inputs are simulated or prepared. The engine logic itself should run for real. This is the cleanest division between what cannot realistically be integrated and what the team can genuinely demonstrate.

Input / dependency	Hackathon treatment	Production direction
Sunshine Cell topology	Synthetic DAPTO-01	DNSP topology / transformer / phase data
Solar/export forecast	Prepared or simulated intervals	Forecast + smart-meter/inverter data
Network condition / limit	Simulated	DNSP telemetry and operating constraints

Input / dependency	Hackathon treatment	Production direction
Household meter readings	Simulated	Authorised interval-meter access
Hot-water compatibility	Mock partner response	Retailer/metering-provider/ aggregator check
Device dispatch	Simulated	Authorised control API
Community battery response	Simulated	Aggregator/DNSP interface
Retail settlement	Calculated ledger only	Retailer/program settlement integration
Optimiser / M&V / policy logic	REAL CODE	Same logic, progressively validated

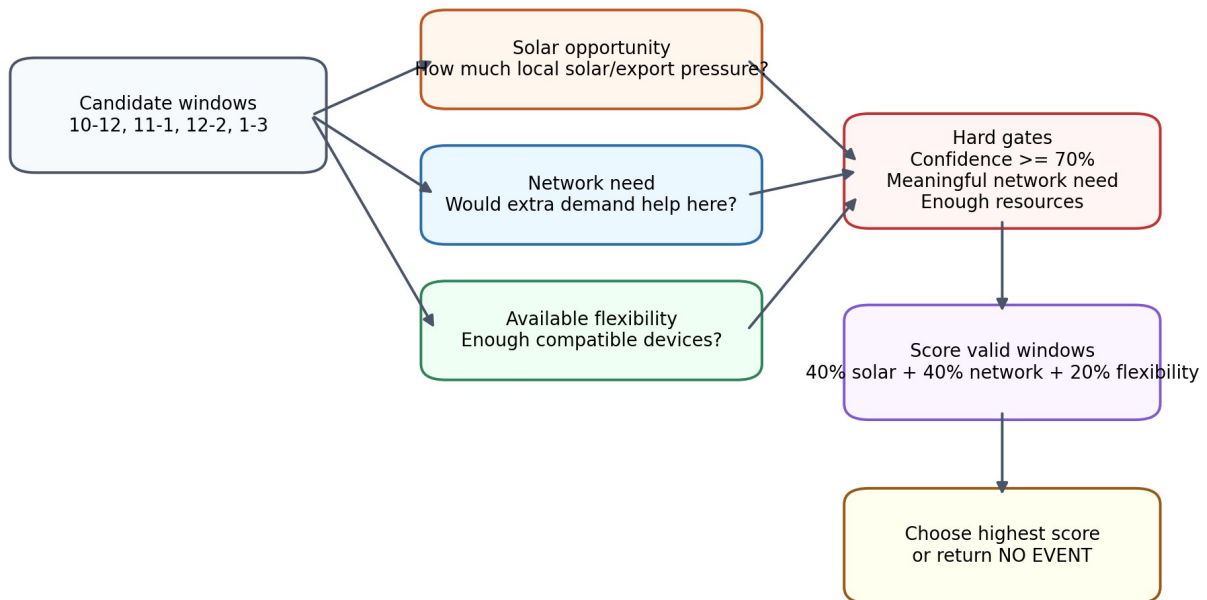
Data-honesty rule

Every simulated value should be labelled SIMULATED or DEMO. Every approximate network mapping should be labelled APPROXIMATE / SYNTHETIC. Do not hide the lack of live industry integrations.

4. Algorithm 1 - Event-window selection

Purpose: choose when a Sunshine Event should happen. The engine looks for the time where solar opportunity, network need and available flexible demand overlap.

How the Engine Chooses the Event Window



4.1 Candidate windows

For the MVP, do not scan every possible minute. Use a small set of overlapping candidate windows such as 10:00-12:00, 11:00-13:00, 12:00-14:00 and 13:00-15:00.

4.2 Hard gates

- Forecast confidence must meet the prototype threshold (for example, $\geq 70\%$).
- Network need must be meaningful; otherwise there is no reason to dispatch.
- There must be enough eligible flexible energy to make the event worthwhile.
- Any critical customer-comfort or operational condition must be satisfied.

4.3 Window score

```

windowScore = 0.40 * solarOpportunity
              + 0.40 * networkNeed
              + 0.20 * flexibilityAvailability

```

These are prototype weights, not universal scientific constants. They are intentionally simple and explainable for the hackathon.

Window	Solar	Network	Flex availability	Score	Result
10-12	70	55	80	65.0	Valid, lower score
11-1	85	75	90	82.0	Valid
12-2	95	90	85	91.0	CHOSEN
1-3	80	70	60	72.0	Valid

No-event state

If no window passes the gates, the engine returns NO EVENT TODAY. This is a credibility feature: the system should not manufacture an event every day.

5. How the target flexibility is calculated

The target (for example 24 kWh) should be an output of the simulated local network opportunity, not a magic number typed into the UI.

```

forecastExportEnergyKwh      = 70
comfortableExportEnergyKwh   = 46

targetFlexEnergyKwh = max(0, 70 - 46)
                    = 24 kWh

```

Interpretation: during the selected window, the synthetic cell is forecast to have about 70 kWh of export pressure, while the demo network model says 46 kWh can be comfortably accommodated. The engine therefore seeks approximately 24 kWh of additional flexible demand.

Unit discipline

kW = power at an instant. kWh = energy over a time period. The event target is energy, so name the field targetFlexEnergyKwh. Device maxPowerKw is a different constraint.

Field	Meaning	Example
targetFlexEnergyKwh	Total flexible energy the event seeks	24 kWh

Field	Meaning	Example
maxPowerKw	Maximum instantaneous device power	3.6 kW hot-water element
maxShiftEnergyKwh	Maximum energy the device may shift in this event	5 kWh

6. Algorithm 2 - Resource eligibility gate

Purpose: reject resources that are not allowed to participate before any ranking score is calculated. A high equity score must never override a technical or safety failure.

```

if consentStatus != 'ACTIVE': reject('No consent')
if resource.sunshineCellId != event.sunshineCellId: reject('Wrong Sunshine Cell')
if resource.available == false: reject('Unavailable')
if resource.compatible == false: reject('Incompatible device')
if resource.comfortSafe == false: reject('Comfort constraint')
else: eligible

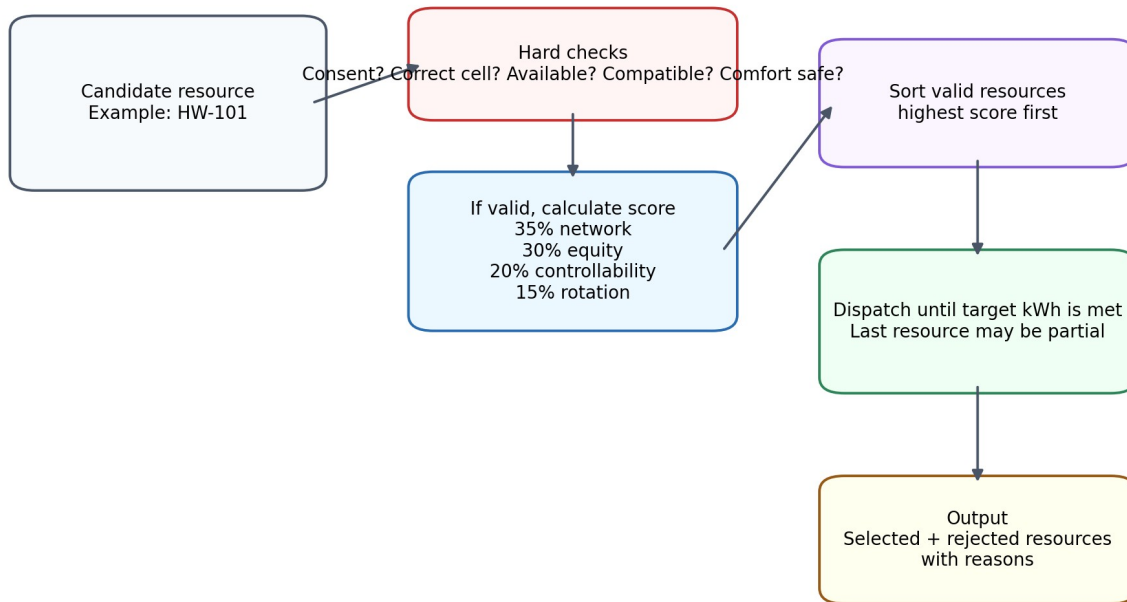
```

Resource	Checks	Decision
HW-101	Correct cell, available, compatible, consent active, comfort safe	ELIGIBLE
EV-04	Available = false	REJECT - unavailable
HW-205	Cell = DAPTO-02 while event is DAPTO-01	REJECT - wrong Sunshine Cell
HW-310	Customer paused participation	REJECT - no active consent

For the hackathon, compatibility can be returned by a simulated authorised-partner endpoint. If the mock partner says compatible=false, the rest of the app must genuinely treat that device as ineligible.

7. Algorithm 3 - Resource scoring and optimisation

Resource Optimiser - Filter First, Rank Second



7.1 Prototype score

$$\begin{aligned}
 \text{resourceScore} = & 0.35 * \text{networkEffectiveness} \\
 & + 0.30 * \text{equityNeed} \\
 & + 0.20 * \text{controllability} \\
 & + 0.15 * \text{rotationFairness}
 \end{aligned}$$

Meaning of the four components:

Criterion	Plain-English meaning
Network effectiveness (35%)	How useful this resource is for the particular local network need.
Equity need (30%)	How strongly using this resource supports the equity objective.
Controllability (20%)	How reliably the resource can be scheduled or controlled.
Rotation fairness (15%)	Whether this participant has been used/rewarded less often than others.

7.2 Example calculation

HW-101
 Network effectiveness = 95
 Equity need = 90
 Controllability = 100
 Rotation fairness = 80

$$\begin{aligned}\text{Score} &= 95 * 0.35 + 90 * 0.30 + 100 * 0.20 + 80 * 0.15 \\ &= 92.25 / 100\end{aligned}$$

7.3 Greedy selection for the MVP

After sorting eligible resources from highest score to lowest score, dispatch resources until the target energy is met. The final resource may be partially dispatched.

Resource	Score	Max shift	Selected
HW-101	92.25	5 kWh	5 kWh
HW-102	86.00	4 kWh	4 kWh
BAT-01	78.00	10 kWh	10 kWh
APT-01	76.00	6 kWh	5 kWh

Target = 24 kWh
 5 + 4 + 10 = 19 kWh
 Remaining = 5 kWh
 APT-01 can shift up to 6 kWh, so dispatch only 5 kWh.
 Final planned dispatch = 24 kWh.

What the score does NOT do

The score decides selection order. It does not determine how many kWh a resource contributes. The dispatched energy is limited by maxShiftEnergyKwh, power/time limits and the remaining target.

8. Algorithm 4 - Deterministic event simulation

Purpose: represent what happens after the optimiser issues a dispatch plan. Because real device control is unavailable during the hackathon, the app uses deterministic response factors.

8.1 Deterministic response factor

A deterministic response factor is a fixed multiplier between 0 and 1 representing how much of the scheduled energy the simulated resource actually delivers. Deterministic means repeatable: the same demo inputs always give the same result.

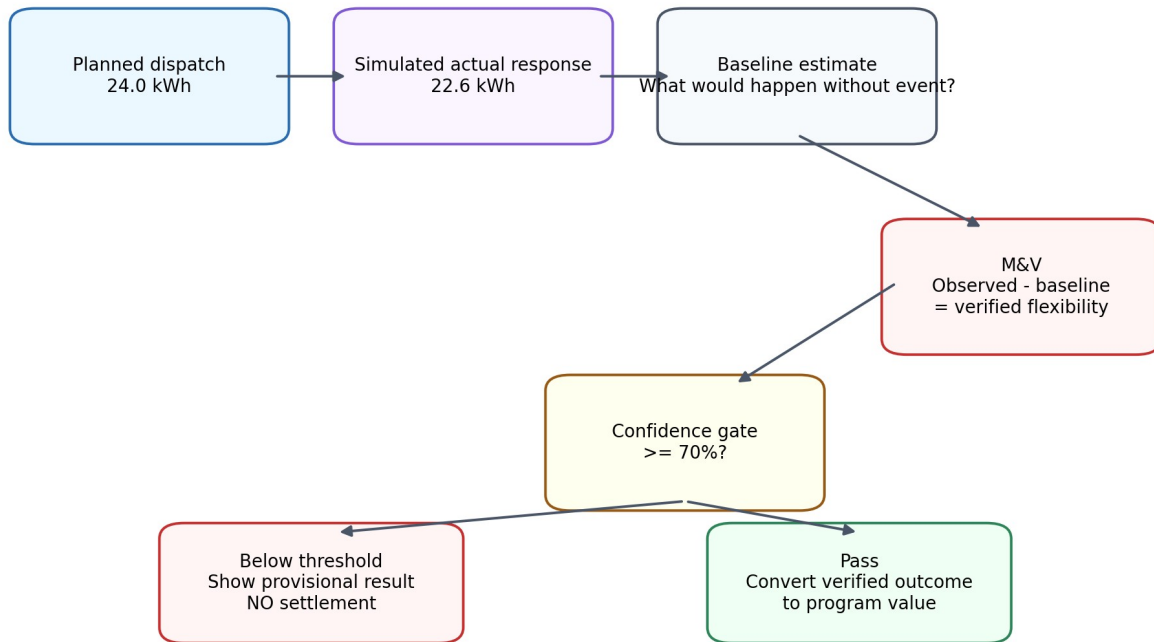
$$\text{actualResponseKwh} = \text{scheduledEnergyKwh} * \text{responseFactor}$$

Resource	Scheduled	Response factor	Simulated actual
HW-101	5.0 kWh	0.96	4.80 kWh
HW-102	4.0 kWh	0.975	3.90 kWh
BAT-01	10.0 kWh	0.97	9.70 kWh
APT-01	5.0 kWh	0.84	4.20 kWh
TOTAL	24.0 kWh	-	22.60 kWh

Why not simply return 24 kWh? Because scheduled response and delivered response are not identical in real systems. The deterministic factors create a believable difference without making the demo random or fragile.

9. Algorithm 5 - Measurement & Verification (M&V)

From Planned Dispatch to Verified Value



Plain-English definition

M&V asks: compared with what would normally have happened, how much extra useful demand did Sunshine Wallet actually create?

9.1 Why M&V exists

The optimiser only says what should happen. M&V checks what actually changed. Credits should not be based on the planned 24 kWh merely because the optimiser scheduled it.

9.2 Core calculation

```
baselineEventEnergyKwh = 50.0
observedEventEnergyKwh = 72.6
```

```
verifiedFlexEnergyKwh = observedEventEnergyKwh - baselineEventEnergyKwh
                      = 22.6 kWh
```

The actual event-period demand is higher because flexible consumption has been moved into the solar-rich period. This does not necessarily mean total daily energy use is higher; the goal is usually to shift timing, such as heating water at midday instead of later.

Important limitation

Higher demand alone does not prove additional solar was physically accommodated. A production M&V method would use richer meter, export and network evidence. In the MVP, the network condition and meter series are synthetic and the calculation is a transparent demonstration.

10. Baseline design in detail

10.1 What is a baseline?

The baseline is the best estimate of what electricity use would have been during the event window if Sunshine Wallet had done nothing.

Comparable non-event days, 12:00-14:00

Day A = 49 kWh

Day B = 51 kWh

Day C = 50 kWh

Baseline = average(49, 51, 50) = 50 kWh

10.2 Why event days must be excluded

If previous Sunshine Wallet event days are included, the baseline drifts upward and the current event appears to create less value than it really did.

BAD baseline if event days are included:

50 (no event) + 72 (event) + 70 (event) = 192

192 / 3 = 64 kWh

Current actual = 72 kWh

72 - 64 = only 8 kWh apparent response

For the MVP, baseline reference days should therefore be comparable days where eventRan=false.

```
referenceDays = historicalDays
  .filter(day => day.eventRan === false)
  .filter(day => isComparable(day, eventDay))
```

```
baseline = average(referenceDays.map(day => day.eventPeriodEnergyKwh))
```

10.3 If too few clean days exist

- Lower the M&V confidence.
- Return a provisional result and withhold settlement.
- In production, use a validated baseline methodology incorporating weekday, weather, temperature, recent demand and other relevant factors.

11. M&V confidence and settlement gate

Confidence is a safety/quality gate. The exact percentage is a prototype policy, not an industry standard. It answers: are we confident enough in the forecast/baseline/measurement to settle money?

MIN_MV_CONFIDENCE = 0.70

```
if mvConfidence < MIN_MV_CONFIDENCE:
    status = 'PROVISIONAL'
    allowSettlement = false
else:
    status = 'VERIFIED'
    allowSettlement = true
```

Verified result	M&V confidence	Decision
22.6 kWh	82%	Settle allowed
18.4 kWh	63%	Show provisional result; no

Verified result	M&V confidence	Decision
		settlement

There can be two separate confidence gates: a forecast-confidence gate before dispatch, and an M&V-confidence gate after the event before financial settlement.

12. Algorithm 6 - Contributor attribution

Purpose: determine how contributor rewards are divided without claiming that Maria's exact electrons flowed to Dinh.

12.1 Measure qualifying export during the event

Maria: 6.5 kWh
 Alex: 4.0 kWh
 Sam: 2.5 kWh
 Total qualifying export = 13.0 kWh

12.2 Calculate contributor shares

Maria share = $6.5 / 13.0 = 50\%$
 Alex share = $4.0 / 13.0 = 30.77\%$
 Sam share = $2.5 / 13.0 = 19.23\%$

12.3 Cap attribution at the verified accommodated result

Do not reward all 13 kWh as though the program definitely accommodated all of it if only 10 kWh of additional solar accommodation is verified in the demo accounting.

```
totalAttributableKwh = min(totalQualifyingExportsKwh, verifiedAccommodatedKwh)
                      = min(13, 10)
                      = 10 kWh
```

Maria attributed kWh = $50\% * 10 = 5$ kWh

What this means

Maria is attributed 50% of the verified contributor outcome for settlement. It does NOT mean 5 kWh of Maria's exact electrons went to Dinh.

13. Algorithm 7 - Settlement and Equity Floor

13.1 Convert verified outcome into prototype program value

For the hackathon, use a clearly labelled demo settlement rate. It is not a real tariff.

verifiedFlexEnergyKwh = 22.6
 prototypeValueRate = \$0.80 per verified kWh

programValue = $22.6 * 0.80 = \$18.08$

13.2 Split into policy pools

Policy:
 65% Equity Pool
 30% Contributor Pool

5% Community Reserve

$\$18.08 * 0.65 = \11.75 equity
 $\$18.08 * 0.30 = \5.42 contributors
 $\$18.08 * 0.05 = \0.90 reserve

13.3 Enforce the Equity Floor

The Equity Floor is a minimum share of distributable program value that must reach qualifying equity participants. It is about money/value, not about requiring 60% of the 24 kWh to come from disadvantaged households.

```

equityFloor = 0.60
proposedEquityShare = 0.65

if proposedEquityShare < equityFloor:
    reject policy / settlement
else:
    settlement valid
  
```

Policy example	Equity	Contributors	Reserve	Valid?
A	65%	30%	5%	YES - 65% >= 60%
B	50%	45%	5%	NO - 50% < 60%

13.4 Split the Equity Pool

For the MVP, keep receiver allocation simple and transparent. Example predefined equity weights:

```

Equity Pool = $11.75
Dinh = 35% -> $4.11 bill/program credit
Sarah = 35% -> $4.11
James = 30% -> $3.53
  
```

The receiver benefit should be described as a bill/program credit that lowers the effective electricity bill. Do not double-count it as both naturally cheaper electricity and a second cash reward unless the tariff design explicitly supports both.

13.5 Split the Contributor Pool

```

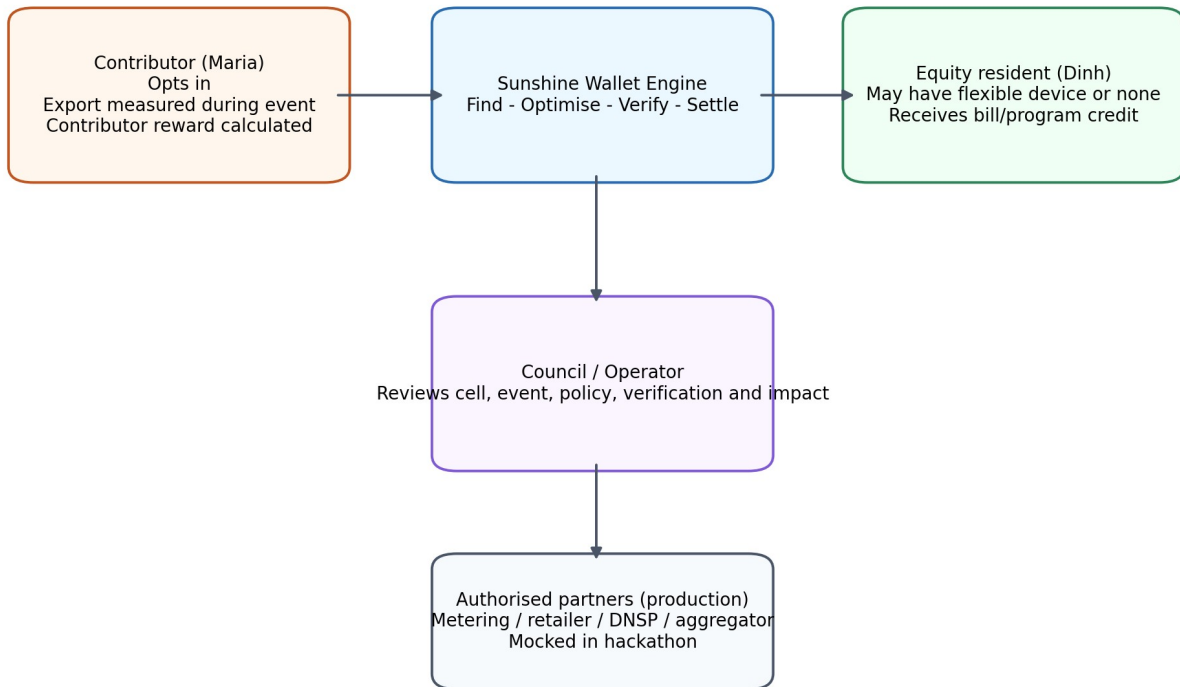
Contributor Pool = $5.42
Maria share = 50%
Maria reward = $5.42 * 0.50 = $2.71
  
```

Economic story

Receivers get a Flexibility Dividend / bill credit. Contributors get a separate Contributor Reward. The exact commercial source of program value must be treated as a pilot/partner settlement assumption in the hackathon.

14. Resident, contributor and Council flows

How Council, Contributors and Equity Residents Connect



14.1 Council / operator flow

1. Open DAPTO-01 Sunshine Cell and review the predicted event opportunity.
2. See the chosen window, target flexible energy, available flexibility and forecast confidence.
3. Run the optimiser and inspect selected/rejected resources with reasons.
4. Review the Equity Floor / policy settings.
5. Click Simulate Flex Event.
6. Review before/after demand, verified flexibility and M&V confidence.
7. If confidence passes the threshold, review settlement pools and apply demo credits.
8. Return to resident/contributor wallets to close the loop.

14.2 Equity resident flow - Dinh

9. Join Sunshine Wallet and pass a simulated program/eligibility check.
10. Optionally register a flexible appliance such as storage hot water; if no compatible device exists, remain an Equity Pool participant.
11. Give consent and keep the ability to pause/opt out.
12. After a verified event, receive a Flexibility Dividend / program credit.
13. Wallet explains the event, credit and reason in simple language.

14.3 Contributor flow - Maria

14. Opt in as a solar/flexibility contributor.
15. Keep participation Active; optionally skip a specific upcoming event.
16. During a qualifying event in the correct Sunshine Cell, simulated meter data records export/contribution.
17. Contributor-attribution logic calculates Maria's proportional share of the verified outcome.
18. Contributor Pool is divided and Maria receives a Contributor Reward.

Do not say "give energy away"

Use language such as "Participate in local energy events" or "Contributor Reward". Maria is not donating traceable electrons to Dinh.

15. Recommended MVP data model

For a 25-hour build, a database is optional. These domain objects can live in TypeScript fixtures/JSON and later map cleanly to tables.

Entity	Important fields
Household	id, displayName, sunshineCellId, householdType, equityEligible, participationMode, consentStatus, walletBalance
Resource	id, householdId?, sunshineCellId, type, compatible, available, consentStatus, comfortSafe, maxPowerKw, maxShiftEnergyKwh, responseFactor, scoringInputs
ContributorMeterInterval	contributorId, eventId, generationKwh, householdConsumptionKwh, exportKwh, qualifyingExportKwh
SunshineCell	id, name, suburb, synthetic flag, equityScore, confidence metadata
EnergyInterval	timestamp, baselineDemandKw, observedDemandKw, forecastSolarKw, exportPressureKw/network proxy
FlexEvent	id, sunshineCellId, startTime, endTime, targetFlexEnergyKwh, forecastConfidence, status
Dispatch	eventId, resourceId, selectedEnergyKwh, simulatedActualEnergyKwh, decisionReason, score
Verification	eventId, baselineEventEnergyKwh, observedEventEnergyKwh, verifiedFlexEnergyKwh, mvConfidence, allowSettlement
Settlement	eventId, programValue, equityPool, contributorPool, reserve, policyVersion
Credit	eventId, householdId, type, amount, explanation

15.1 TypeScript example

```
type Resource = {
  id: string;
  householdId?: string;
  sunshineCellId: string;
  type: 'HOT_WATER' | 'BATTERY_CHARGE' | 'EV_CHARGE' | 'APARTMENT_LOAD';
  compatible: boolean;
  available: boolean;
  consentStatus: 'ACTIVE' | 'PAUSED';
  comfortSafe: boolean;
  maxPowerKw: number;
  maxShiftEnergyKwh: number;
  responseFactor: number;
```

```

    networkEffectiveness: number; // 0-100
    equityNeed: number;           // 0-100
    controllability: number;       // 0-100
    rotationFairness: number;      // 0-100
};

type FlexEvent = {
  id: string;
  sunshineCellId: string;
  startTime: string;
  endTime: string;
  forecastExportEnergyKwh: number;
  comfortableExportEnergyKwh: number;
  targetFlexEnergyKwh: number;
  forecastConfidence: number;
};

```

16. JSON / API contract examples

16.1 Demo state

```

GET /api/demo/state
{
  "cell": {
    "id": "DAPTO-01",
    "name": "Dapto Sunshine Cell",
    "synthetic": true
  },
  "event": {
    "id": "EVT-001",
    "startTime": "2026-08-23T12:00:00+10:00",
    "endTime": "2026-08-23T14:00:00+10:00",
    "forecastExportEnergyKwh": 70,
    "comfortableExportEnergyKwh": 46,
    "targetFlexEnergyKwh": 24,
    "availableFlexEnergyKwh": 31,
    "forecastConfidence": 0.82,
    "status": "PLANNED"
  }
}

```

16.2 Optimise

```

POST /api/events/EVT-001/optimise
{
  "equityFloor": 0.60
}

RESPONSE
{
  "targetFlexEnergyKwh": 24,
  "selected": [
    {"resourceId": "HW-101", "score": 92.25, "selectedEnergyKwh": 5},
    {"resourceId": "HW-102", "score": 86.00, "selectedEnergyKwh": 4},
    {"resourceId": "BAT-01", "score": 78.00, "selectedEnergyKwh": 10},
    {"resourceId": "APT-01", "score": 76.00, "selectedEnergyKwh": 5}
  ],
  "rejected": [
    {"resourceId": "EV-04", "reason": "UNAVAILABLE"},
    {"resourceId": "HW-205", "reason": "WRONG_SUNSHINE_CELL"}
  ],
}

```



```
"plannedFlexEnergyKwh": 24
}
```

16.3 Run and verify

POST /api/events/EVT-001/run-and-verify

RESPONSE

```
{
  "simulation": {
    "plannedFlexEnergyKwh": 24,
    "simulatedDeliveredFlexEnergyKwh": 22.6
  },
  "verification": {
    "baselineEventEnergyKwh": 50.0,
    "observedEventEnergyKwh": 72.6,
    "verifiedFlexEnergyKwh": 22.6,
    "mvConfidence": 0.82,
    "allowSettlement": true
  }
}
```

16.4 Settle

POST /api/events/EVT-001/settle

```
{
  "equityFloor": 0.60,
  "equityShare": 0.65,
  "contributorShare": 0.30,
  "reserveShare": 0.05,
  "prototypeValueRatePerKwh": 0.80
}
```

RESPONSE

```
{
  "verifiedProgramValue": 18.08,
  "equityPool": 11.75,
  "contributorPool": 5.42,
  "communityReserve": 0.90,
  "credits": [
    {"participantId": "DINH", "type": "EQUITY_DIVIDEND", "amount": 4.11},
    {"participantId": "MARIA", "type": "CONTRIBUTOR_REWARD", "amount": 2.71}
  ],
  "policyValid": true
}
```

16.5 Reset endpoint for demo reliability

POST /api/demo/reset

Purpose: reset the hero scenario to a known starting state before each judging run.

17. Coordinator function and pseudocode

Keep the engine as pure functions where possible. The UI calls one coordinator that orchestrates them.

```
function runSunshineEvent(input) {
  const window = selectEventWindow(input.forecast, input.network, input.resources);
  if (!window) return { status: 'NO_EVENT' };

  const targetFlexEnergyKwh = calculateTargetFlexEnergy(window);
```

```

const { eligible, rejected } = checkResourceEligibility(
  input.resources,
  window
);

const dispatchPlan = optimiseResources(
  eligible,
  targetFlexEnergyKwh
);

const simulation = simulateEvent(
  dispatchPlan,
  input.energySeries
);

const baseline = calculateBaseline(
  input.historicalDays.filter(d => !d.eventRan)
);

const verification = verifyEvent(
  baseline,
  simulation,
  input.mvInputs
);

if (!verification.allowSettlement) {
  return {
    status: 'PROVISIONAL',
    rejected,
    dispatchPlan,
    simulation,
    verification
  };
}

const attribution = attributeContributors(
  input.contributorExports,
  verification
);

const settlement = settleEvent(
  verification,
  attribution,
  input.policy
);

return {
  status: 'SETTLED',
  rejected,
  dispatchPlan,
  simulation,
  verification,
  attribution,
  settlement
};
}

```

18. Test scenarios the team should implement

Test	Input change	Expected result
Happy path	82% forecast confidence, 24 kWh target, enough resources	Event runs, verifies, settles
Low forecast confidence	Forecast confidence = 55%	NO EVENT
Wrong cell	HW-205 in DAPTO-02	Rejected before scoring
Unavailable EV	EV-04 available=false	Rejected before scoring
Partial final dispatch	19 kWh selected, next resource max 6, target 24	Dispatch next resource for 5 kWh only
Delivered < planned	24 kWh planned, response factors produce 22.6	M&V uses actual/observed result
Contaminated history	Historical day eventRan=true	Excluded from baseline
Low M&V confidence	mvConfidence = 0.63	Provisional result; no settlement
Invalid equity policy	Equity share 50%, floor 60%	Reject policy
Maria export changes	Maria export drops from 6.5 to 2 kWh	Her contributor share/reward falls
No flexible device for Dinh	Dinh has no compatible resource	Can remain Equity Pool participant

19. Known limitations and defensible judge language

Possible challenge	Defensible response
"Is the feeder/topology real?"	No. DAPTO-01 is a clearly labelled synthetic Sunshine Cell for the prototype. Production requires DNSP topology/telemetry.
"Are you controlling real hot water?"	No. Partner control is mocked. The app genuinely processes compatibility, consent, dispatch, verification and settlement logic.
"Can you prove Maria's electrons reached Dinh?"	No, and we do not claim that. We use event-window metering/accounting attribution inside the relevant cell.
"Where does the money come from?"	The MVP uses a simulated settlement rate. A production pilot needs an agreed retailer/network/aggregator program value mechanism.
"Is 70% confidence a standard?"	No. It is a prototype threshold chosen to demonstrate a confidence gate.
"Are 35/30/20/15 optimiser weights scientifically validated?"	No. They are transparent prototype weights for an explainable MVP.
"Does higher midday demand prove solar was rescued?"	Not by itself. The MVP demonstrates the M&V workflow with synthetic inputs; production requires richer export/network evidence.
"Why not reward raw noon consumption?"	That encourages gaming. We use baseline-based verified response instead.

Possible challenge	Defensible response
"What if every historical day has an event?"	Do not contaminate the baseline. Lower confidence or use a validated production baseline model with matched/control periods.

20. Implementation priorities and cheat sheet

20.1 P0 engine functions

- `checkResourceEligibility()` - hard safety/technical/consent filters.
- `optimiseResources()` - score, sort and partially dispatch until target is met.
- `calculateBaseline()` + `verifyEvent()` - baseline-based M&V and confidence gate.
- `settleEvent()` - program value, Equity Floor, receiver credits and contributor rewards.

20.2 P1 engine functions

- `selectEventWindow()` - choose the best window rather than hardcoding noon.
- `calculateTargetFlexEnergy()` - derive target from synthetic network opportunity.
- `attributeContributors()` - proportional contributor accounting capped at verified outcome.
- `simulateEvent()` - deterministic resource response factors.

20.3 Five sentences to remember

19. Window selection asks **WHEN** the event is useful.
20. Eligibility asks **WHO** is allowed to participate.
21. The optimiser asks **WHICH** valid resources should be used and **HOW MUCH** energy to dispatch.
22. M&V asks **WHAT** actually changed compared with what would have happened anyway.
23. Settlement asks **WHO** receives the verified value, with the Equity Floor protecting the equity objective.

20.4 Final worked example

DAPTO-01 HERO SCENARIO

1. Window chosen: 12:00-14:00
Forecast confidence: 82%
2. Network opportunity:
Forecast export = 70 kWh
Comfortable export = 46 kWh
Target flex = 24 kWh
3. Eligibility:
HW-101 eligible
HW-102 eligible
BAT-01 eligible
APT-01 eligible
EV-04 rejected: unavailable
HW-205 rejected: wrong cell
4. Optimiser:
HW-101 5 kWh
HW-102 4 kWh
BAT-01 10 kWh
APT-01 5 kWh
Planned total = 24 kWh

5. Simulation:
Deterministic delivered response = 22.6 kWh
6. M&V:
Baseline = 50.0 kWh
Observed = 72.6 kWh
Verified = 22.6 kWh
M&V confidence = 82% -> settlement allowed
7. Contributor accounting:
Maria = 6.5 / 13.0 qualifying export = 50% share
8. Program value:
 $22.6 * \$0.80 \text{ demo rate} = \18.08
9. Policy:
65% equity = \$11.75
30% contributors = \$5.42
5% reserve = \$0.90
Equity Floor = 60% -> policy valid
10. Example credits:
Dinh equity dividend = \$4.11
Maria contributor reward = \$2.71

Development principle

Build the logic judges need to see change when inputs change. Simulate the external electricity infrastructure, but make the optimiser, verification, attribution, policy enforcement and wallet calculations genuinely execute.

Source basis and design-status notes

Source-derived concept: Sunshine Wallet - A Local Energy Equity Orchestrator for Wollongong (22 August 2026). The source defines Sunshine Cells, flexible-load orchestration, verification, Equity Floor governance, contributor rewards, confidence disclosure, partner-adaptor architecture and the recommendation to simulate unavailable infrastructure in the hackathon.

MVP design decisions refined in discussion: explicit event-window scoring example, target-flexibility derivation example, deterministic response factors, exclusion of prior event days from the simple baseline, contributor attribution formula, example settlement rates and pool percentages, and exact JSON/API shapes. These are prototype implementation choices, not claimed industry standards.

All monetary rates and detailed DAPTO-01 network values in this document are illustrative/simulated for the hackathon unless explicitly replaced by validated partner data.